

Assignment - 04• Sec-A ($5 \times 2 = 10$)

Q1. What is demand paging?

Ans. In OS, demand paging is a method of memory management that enables a program to be executed even if not all of its code and data are in physical memory at the same time. It involves loading data into memory only when it is specifically requested, or "demanded", by a program during its execution. This approach allows for more efficient use of memory resources as only the necessary portions of a program.

Q2. Explain memory management in UNIX.

Ans. In UNIX, memory management involves allocating and deallocating memory for processes. The kernel manages this through mechanisms like virtual memory, utilizing RAM and swap space to optimize performance and ensure efficient use of resources.

Q3. What is multi-level paging?

Ans. Multilevel paging is a memory management scheme used in computer operating systems. Instead of having a single page table to map virtual address to physical address, multilevel paging uses a hierarchy of page tables. This helps manage large address spaces more efficiently.

Multilevel paging is beneficial for systems with large address space, as it enables efficient memory management without the need to store an excessively large and sparse page table.

Q4. Explain various swapping schemes.

Ans. (i) Whole process swapping :- Swaps out entire processes between main memory and secondary storage.

- (ii) Partial swapping: Swaps only a portion of a process allowing more flexibility in memory management.
- (iii) Pager system :- Divides both the process and physical memory into fixed-size pages. Enables more efficient use of available memory.
- (iv) Segmentation :- divides processes into logical segments. Each segment can be swapped independently providing more flexibility in memory allocation.
- (v) Priority based swapping
- (vi) Demand paging
- (vii) Contiguous memory allocation :-
- (viii) Non-contiguous memory allocation.

Q5. What is memory management?

Ans Memory management is a crucial aspect of operating systems that deals with the allocation, deallocation, and utilization of physical memory (RAM) to multiple processes running concurrently. It ensures that each process has access to the memory it needs while preventing conflicts and ensuring efficient use of the available memory resources.

• Section - B ($7 \times 3 = 21$)

Q1. Causes and resolutions for page interrupt faults, their impact, memory management, OS strategies and system stability.

Ans. Page interrupt faults, also known as page faults, are errors that occur when a process attempts to access a memory address that is not currently loaded into physical memory (RAM). These faults are a crucial part of memory management, as they allow the

operating system to efficiently handle memory allocation and utilization.

- Causes of Page Faults :- There are several factors that can contribute to page interrupt faults:

- (1) Demand Paging :- In demand paging memory management, pages are only loaded into physical memory when they are needed. If a process accesses a page that is not yet loaded, a page fault occurs.
- (2) Memory Fragmentation :- When memory becomes fragmented due to frequent allocation and deallocation, it may be difficult to find contiguous memory blocks to satisfy a page fault.
- (3) Memory overallocation :- If the operating system allocates more memory than the available physical memory, it will eventually lead to page faults.
- (4) Hardware Errors :- Hardware malfunctions can also cause page faults, typically due to faulty memory chips or related components.

- Resolution for Page Faults :- When a page fault occurs, the operating system initiates the page fault handling process:

- (1) Page Fault Detection :- The CPU detects an invalid memory address and raises a page fault exception.
- (2) Page Fault Handling :- The OS interrupts the current process and identifies the missing page.
- (3) Page Loading :- The operating system locates the corresponding page on secondary storage (usually a hard disk) and loads it into a free physical memory frame.
- (4) Page Table Updates :- The page table entry for the page is updated to reflect its new location in physical memory.

(5) Process Resumption :- The interrupted process resumes execution with access to the required page.

• Impact of Page Faults on System Performance :-
Page faults can impact system performance in a few ways :

- (1) Context Switching Overhead :- Switching betⁿ processes due to page faults adds overheads and can slow down overall system responsiveness.
- (2) Disk Access Latency :- Loading pages from secondary storage can introduce latency, especially if the disk is slow or heavily loaded.
- (3) Thrashing :- Excessive page faults, known as thrashing can lead to a situation where the OS spends more time swapping pages in and out of memory than executing processes, causing significant performance degradation.

• Memory Management Strategies to minimize Page faults :-

- (1) Demand Paging :- Utilize demand paging to load pages only when needed, reducing the number of page faults and improving memory utilization.
- (2) Effective Page Replacement Policies :- Employ intelligent page replacement algorithms to select pages for eviction from physical memory, ensuring that frequently accessed pages remain in memory.
- (3) Efficient Memory Allocation :- Allocate memory to processes in a way that minimizes fragmentation.

and Improves memory utilization.

- Operating System Strategies for Handling Page Faults:- OS employ various strategies to handle page faults efficiently:-

- (1) Page Fault Anticipation:- Predict potential page faults and pre-load pages into physical memory before they are actually accessed.
- (2) Page Fault Clustering: Batch page faults together to reduce the overhead associated with context switching and disk access.
- (3) Page Fault Logging: Log page faults to identify patterns and optimize memory management strategies.
- (4) Page Fault Handling Optimizations:- Optimize the page fault handling process to minimize its impact on system performance.

Maintaining System Stability:- By effectively handling page faults and employing efficient memory management strategies, operating systems can maintain system stability and prevent performance degradation due to excessive page faults. This ensures that systems can provide a responsive and consistent user experience even under varying workloads.

Q2. Role of virtual memory in efficient memory management, paging, segmentation, and multitasking support.

Sol. Virtual memory plays a crucial role in efficient memory management by providing an illusion to the application that there is more physical

memory (RAM) available than there actually is this is achieved through a combination of paging and segmentation.

(1) Paging :-

Virtual memory allows the OS to divide the physical memory into fixed-size blocks called pages.

- Pages are used as the basic unit of data transfer b/w the disk and RAM.
- When a program needs more memory than is physically available then, the operating system swaps data between RAM and the storage device loading only the necessary pages into RAM.

(2) Segmentation :- Virtual memory supports segmentation dividing a program into logical segments such as code, data and stack.

- Each segment is managed independently allocating for flexible memory allocation and protection.
- Segmentation helps in organizing and securing the memory space for different components of a program.

(3) Multitasking Support :-

Virtual memory enables multitasking by allowing multiple programs to run simultaneously even if they collectively require more memory than the physical RAM can provide.

Virtual address spaces, and the OS manages the swapping of pages in and out of RAM as needed.

Q3. impact and benefits of multi-level paging on memory management, address translation, and system performance.

Ans. Multilevel paging is a hierarchical memory management technique that utilizes multiple levels of page tables to translate virtual addresses into physical addresses. This approach offers several advantages over traditional single-level paging, including reduced memory overhead, faster address translation, and improved system performance.

• Benefits of Multi-Level Paging :- The adoption of multi-level paging offers several notable benefits:

- (1) Reduced Memory Overhead :- Multi-level paging significantly reduces the memory required to store page table entries, optimizing memory usage.
- (2) Faster Address Translation :- The hierarchical structure of multi-level paging enables faster address translation, improving system responsiveness.
- (3) Improved Memory Utilization - Multi-level paging allows for more efficient memory allocation, reducing fragmentation and maximizing memory usage.
- (4) Enhanced Scalability :- Multi-level paging can efficiently handle large virtual address spaces, making it suitable for modern computing systems.

(5) Reduced Page faults Rates :- Multi-Level Paging can reduce page fault rates, minimizing performance disruptions caused by page faults.

• Impact on Memory Management :- Multilevel Paging has a profound impact on memory management, introducing several improvements:

(i) Efficient Page Table Management :- Multi-Level Paging simplifies page table management by dividing the large page table into manageable levels.

(ii) Reduced Memory Allocation Overheads :- The reduced memory overhead associated with multi-level paging allows for more efficient memory allocation and utilization.

(iii) Improved Page Fault Handling :- Multilevel Paging can optimize page fault handling by reducing the time required to locate the missing page table entry.

• Impact on Address Translation :-

(i) Reduce Address Translation Time :- The hierarchical structure of multi-level paging reduces the number of page table accesses required, speeding up address translation.

(ii) Efficient Address Space Management :- Multilevel paging effectively manages large address spaces, enabling efficient address translation for modern systems.

(iii)

(iv)

Reduced Translation Overhead :- Multilevel paging minimizes the overhead associated with address translation, improving overall system performance.

- Impact on System Performance -

- (i) Reduced Latency - The faster address translation and reduced memory overhead contribute to reduced latency.
- (ii) Improve Scalability -
- (iii) Reduced Thrashing -

- PART-C (11x3=33)

Q1. Explain memory fragmentation, differentiating betⁿ internal and external fragmentation. Provide examples of how each type can occur and suggest strategies to mitigate their impact on system performance.

Ans Memory fragmentation is a common issue in memory management systems, where available memory is not contiguous, making it difficult to allocate large contiguous blocks to processes. This can lead to inefficient memory utilization and performance degradation.

- There are two main types of memory fragmentation.

- (i) Internal Fragmentation
- (ii) External Fragmentation

- Differentiating betⁿ internal and external fragmentation.

(i) Internal Fragmentation - Occurs when a process is too large to fit into a single contiguous block of free memory, even though there is enough total free memory to accommodate the process. This can happen due to the allocation of memory blocks in a non-optimal manner.

(ii) External Fragmentation - Occurs when there are small, scattered free memory blocks that are too small to be used by any process, even though

there is enough total free memory to accommodate the process. This can happen due to the allocation and deallocation of memory block over time.

• Strategies to Mitigate memory fragmentation -

Several strategies can be employed to mitigate the impact of memory fragmentation and improve memory utilization.

- (i) **Paging and Segmentation** - These memory management techniques divide memory into fixed-size pages or variable-size segments.
- (ii) **Segment Replacement** - This page replacement algorithm prioritizes pages in shared segments for eviction, reducing external fragmentation by maximizing the likelihood of reclaiming contiguous free blocks.
- (iii) **Swapping** - This technique temporarily moves inactive processes or pages from physical memory to secondary storage.
- (iv) **Memory Defragmentation** - This process consolidates fragmented memory blocks into larger contiguous blocks, improving memory utilization and reducing fragmentation.

Q2. Explain the concept of Segmentation in memory management. Discuss its advantages, potential issues, and how it differs from Paging. Provide examples to illustrate its practical use.

Ans. Segmentation is a memory management technique that divides a program's logical address space into segments. Each segment represents

a logical unit of a Program, such as code, data or the Stack. Segments can be allocated and deallocated independently, providing flexibility in memory management.

Advantages of Segmentation -

- (i) **Flexible Memory Allocation** - Segmentation allows for efficient allocation of memory to logical units of a Program, providing better control over memory usage.
- (ii) **Enhanced Memory Protection** - Segmentation enables stronger memory protection by isolating segments from each other, preventing unauthorized access or modification of data.
- (iii) **Efficient Memory Sharing** - Segmentation facilitates memory sharing between processes, allowing multiple processes to access and modify common data segments.
- (iv) **Reduced Memory Overhead** - Segmentation typically requires less memory overhead compared to paging, as it doesn't require page table entries for each segment.

Potential issues with Segmentation -

- (i) **External Fragmentation** - Segmentation can lead to external fragmentation, where unused memory fragments are too small to be utilized by any process.
- (ii) **Complex Address Translation** - Segments addresses need to be translated into physical addresses, adding a layer of complexity to address translation.
- (iii) **Limited Scalability** - Segmentation may not scale well to very large address spaces due to the increased overhead of managing variable-sized segments.

Segmentation Vs. Paging -

Feature	Segmentation	Paging
• Memory division	variable-sized segments	Fixed-sized pages
• Address Translation	segment tables	page tables
• Flexibility	more flexible memory allocation	uniform memory allocation
• Memory protection	stronger memory protection	weaker memory protection
• Memory sharing	efficient memory sharing	limited memory sharing
• Memory overhead	lower memory overhead	higher memory overhead

• Examples of practical use -

→ Segmentation is commonly used in OS that support multitasking and memory sharing. Examples include -

- UNIX - Unix utilize segmentation to isolate the memory spaces of multiple processes, preventing conflicts and ensuring data integrity.
- Linux - Linux employs segmentation to manage the memory of processes and shared libraries, enabling efficient data integrity, memory shared and code reuse.

Q3. Consider a page replacement algorithm with a reference string of length 20 and a fixed number of frames set to 3. Using the reference string "7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1," analyze and compare the performance of the FIFO (First in - First out), LRU (Least Recently used) and optimal page replacement algorithms. Calculate the no. of page faults for each algorithm and discuss their strengths and weaknesses.

FIFO :-

F3			1	1	1	1	0	0	0	3	3	3	2	2	2	2	2	7
F2		0	0	0	0	3	3	3	2	2	2	2	1	1	1	1	0	0
F1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	7	7	7	7
H/F	F	F	F	F	H	F	F	F	F	F	F	F	F	H	H	F	F	F

$$\text{Hit} = 5$$

$$\text{Fault} = 15$$

Strength - Simple to implement

Weakness - doesn't always perform for all access pattern.

• LRU -

F3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7
F2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0
F1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1
H/F	F	F	F	F	H	F	H	F	F	F	F	H	H	F	H	F	H	F	H

$$\text{Hit} = 8$$

$$\text{Fault} = 12$$

Strength - tends to perform well in capturing temporal locality.

Weakness - Implementation can be more complex may not always predict best page to replace.

Optimal :-

F3			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1
F2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0
F1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7
H/F	F	F	F	F	H	F	H	F	H	H	F	H	H	F	H	H	F	H	H

$$\text{fault} = 9$$

$$\text{hit} = 11$$

Strength - Provide optimal page replacement strategy by replacing page that will not be used for longest time.

Weakness - not practical to implement in real system as it require knowledge of future page access.

~ ~ ~ ~ ~

[Signature]

3

4